جامعة أبوظبي
Abu Dhabi University

Abu Dhabi University

College of Engineering

ITE 408: Information Security

Lab 3 Report

Part A: Access Control in a Linux System
Part B: Getting Started with Keycloak Identity Provider

Student Name	Anas Syed
Student ID	1091080
Course	ITE 408 – Information Security
Semester	Fall 2026-27
Submission Date	8th November 2026
Total Marks	5 (Part A: 2.5, Part B: 2.5)

Individual Submission

Table of Contents

1. Introduction

2. Part A – Access Control in a Linux System

2.1 Part 1 – Adding a New Group

2.2 Part 2 – Adding and Verifying Users, Groups and Passwords

2.3 Part 3 – Symbolic Permissions

2.4 Part 4 – Absolute (Octal) Permissions

2.5 Part 5 – SUID, SGID and Sticky Bit (Self-Learning)

2.6 Summary of Special Permission Bits

2.7 Critical Analysis – Part A

3. Part B – Getting Started with Keycloak Identity Provider

Steps 1–10

3.1 Critical Analysis – Part B

4. Conclusion

References

1. Introduction

Access control decides **who** can do **what** with **which** resource. This lab looks at two levels of it. **Part A** is about the operating system: on Linux, every file and folder has an owner, a group, and permissions for the user, the group and everyone else. **Part B** is about applications: it uses Keycloak, a free tool that logs users in and hands out secure login tokens (OAuth 2.0 and OpenID Connect).

1.1 Environment

- **Operating system:** Ubuntu 24.04 LTS, used instead of the CSE-LABVM (the lab allows an Ubuntu machine).
- **Prompt:** the terminal prompt was set with `PS1="\d \@ [1091080]"`, so every screenshot shows the date, the time and my student ID. After the ID the prompt also shows the user name and the folder, so you can see who ran each command.
- **Switching users:** the lab uses the desktop "Switch User" menu. I used `su - sam` and `su - danny` in the terminal instead. The result is the same: each command runs with that user's ID and groups.

2. Part A – Access Control in a Linux System

2.1 Part 1 – Adding a New Group

Step 1.1: Prompt changed to show student ID and privileges escalated to root

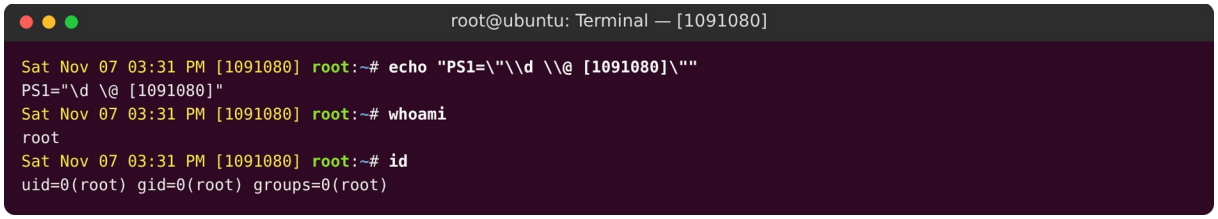


Figure 1: Prompt changed to show student ID and privileges escalated to root

`sudo su` makes the session run as root (user ID 0). Adding groups and users changes system files (`/etc/group`, `/etc/passwd`, `/etc/shadow`), so only root can do it. `id` shows `uid=0(root)`.

Good practice: use `sudo <command>` for one admin task instead of staying in a root shell. It lowers the chance of mistakes and keeps a record in `/var/log/auth.log`.

Step 1.2: Add the DEVELOPERS group

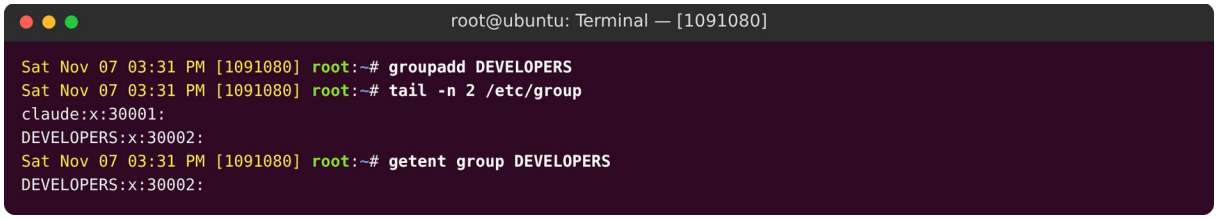


Figure 2: Add the DEVELOPERS group

`groupadd DEVELOPERS` creates the group. The last line of `/etc/group` shows `DEVELOPERS:x:<GID>:`. The fields are: group name, a password placeholder (x), the group ID, and the list of members (empty for now). `getent` reads the same information.

2.2 Part 2 – Adding and Verifying Users, Groups and Passwords

Step 2.1: Add user sam (password samPass) and place him in DEVELOPERS



Figure 3: Add user sam (password samPass) and place him in DEVELOPERS

`adduser` is an easy tool for creating users on Ubuntu. It makes a private group with the same name, makes a home folder, copies the starter files from `/etc/skel` and saves the full name. I set the password `samPass` with `chpasswd`, which gives the same result as typing it when asked.

Correction to the lab handout: the handout uses `usermod -G DEVELOPERS sam`. The `-G` option **replaces** all other groups. I used `usermod -aG`, which **adds** the group and keeps the others. `id sam` shows that DEVELOPERS was added.

Step 2.2: Add user danny (password dannyPass) and place him in DEVELOPERS

```
root@ubuntu: Terminal — [1091080]

Sat Nov 07 03:31 PM [1091080] root:~# adduser --gecos 'Danny,,,' --disabled-password danny
info: Adding user `danny' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `danny' (1002) ...
info: Adding new user `danny' (1002) with group `danny (1002)' ...
info: Creating home directory `/home/danny' ...
info: Copying files from `/etc/skel' ...
info: Adding new user `danny' to supplemental / extra groups `users' ...
info: Adding user `danny' to group `users' ...
Sat Nov 07 03:31 PM [1091080] root:~# echo 'danny:dannyPass' | chpasswd && echo 'passwd: password updated successfully'
passwd: password updated successfully
Sat Nov 07 03:31 PM [1091080] root:~# usermod -aG DEVELOPERS danny
Sat Nov 07 03:31 PM [1091080] root:~# id danny
uid=1002(danny) gid=1002(danny) groups=1002(danny),100(users),30002(DEVELOPERS)
```

Figure 4: Add user danny (password dannyPass) and place him in DEVELOPERS

The same steps were done for danny with the password `dannyPass`. Each user gets its own ID (1001 for sam, 1002 for danny) and a private group.

Step 2.3: Verify the new users in /etc/passwd and /etc/group

```
root@ubuntu: Terminal — [1091080]

Sat Nov 07 03:31 PM [1091080] root:~# tail -n 3 /etc/passwd
claude:x:999:30001::/home/claude:/bin/bash
sam:x:1001:1001:Sam,,,:/home/sam:/bin/bash
danny:x:1002:1002:Danny,,,:/home/danny:/bin/bash
Sat Nov 07 03:31 PM [1091080] root:~# grep DEVELOPERS /etc/group
DEVELOPERS:x:30002:sam,danny
```

Figure 5: Verify the new users in /etc/passwd and /etc/group

`/etc/passwd` has 7 fields separated by colons: user name, `x` (the password is kept in the shadow file), user ID, group ID, full name, home folder and login shell. `/etc/group` now lists `sam,danny` as members of DEVELOPERS.

Step 2.4: Verify the password hashes in /etc/shadow

```
root@ubuntu: Terminal — [1091080]

Sat Nov 07 03:31 PM [1091080] root:~# grep -E '^(sam|danny):' /etc/shadow
sam:$y$j9T$K0GEV8L9ri4jzIuSd6IFP0$srtHi2JJ1LECfyZKeW.cwRr1PgQzi01pJyGdVSDnzS6:20764:0:99999:7:::
danny:$y$j9T$bMbUnKEKekTta5jGvn3Kv0$jbRnBKzo6o6ioB3VJS3eehPOtPx6VLaFnXp8jXLd9Q.:20764:0:99999:7:::
Sat Nov 07 03:31 PM [1091080] root:~# ls -l /etc/shadow /etc/passwd
-rw-r--r-- 1 root root   3425 Nov  7 15:31 /etc/passwd
-rw-r----- 1 root shadow 1575 Nov  7 15:31 /etc/shadow
```

Figure 6: Verify the password hashes in /etc/shadow

`/etc/shadow` stores password **hashes**, not the real passwords. The start `$y$` means the hash type is **yescrypt**, the default on Ubuntu 22.04 and newer (the handout shows `$6$`, an older type called SHA-512). Every hash uses a random salt, so the same password gives a different hash each time. The other fields are: day of the last change (counted from 1970), minimum age 0, maximum age 99999 and warning time 7 days.

Look at the permissions: everyone can read `/etc/passwd` (`-rw-r--r--`), but `/etc/shadow` is `-rw-r-----` and belongs to root and the group shadow. Normal users cannot read the hashes, so they cannot try to crack them offline.

2.3 Part 3 – Symbolic Permissions

Step 3.0: Default home-directory mode on Ubuntu 24.04 (HOME_MODE) and lab baseline

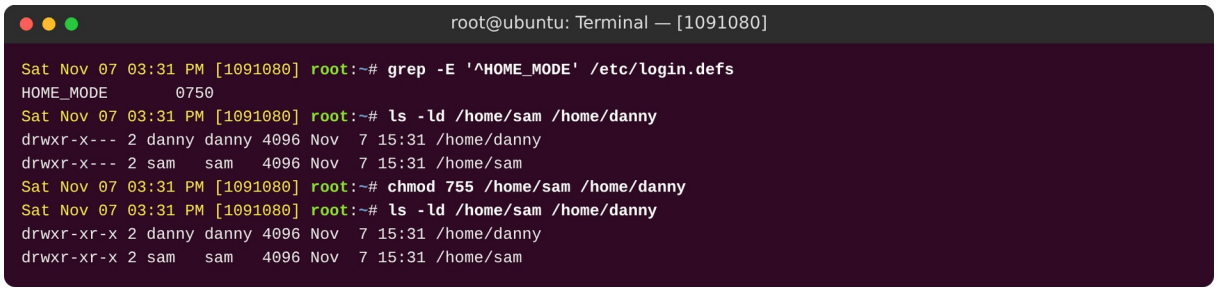


Figure 7: Default home-directory mode on Ubuntu 24.04 (HOME_MODE) and lab baseline

Finding: Ubuntu 21.04 and newer create home folders with mode `0750` (`drwxr-x---`), so other users cannot enter them. This is safer than the `drwxr-xr-x` shown in the handout. To repeat the lab situation, I first set both home folders to 755.

Step 3.1: Switch to sam and explore the environment

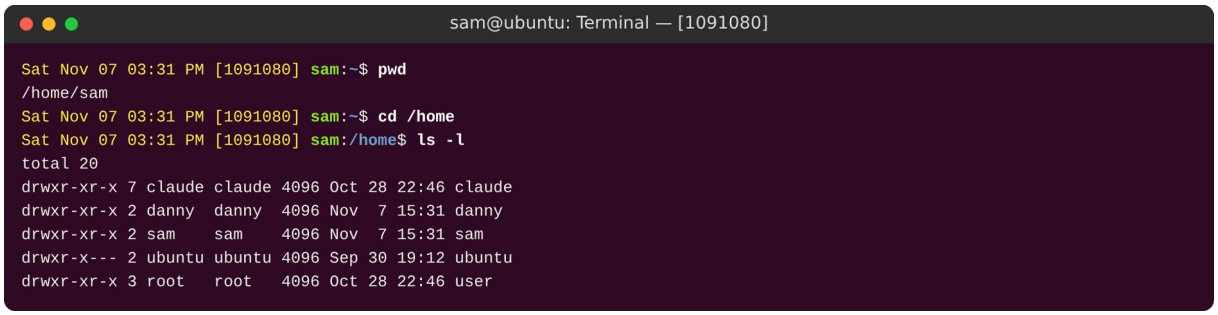


Figure 8: Switch to sam and explore the environment

As sam, `ls -l /home` shows a 10-character permission string. Character 1: `d` means directory. Characters 2 to 4: owner (`rwx`). Characters 5 to 7: group (`r-x`). Characters 8 to 10: others (`r-x`).

Step 3.2: sam can enter danny's directory but cannot write in it

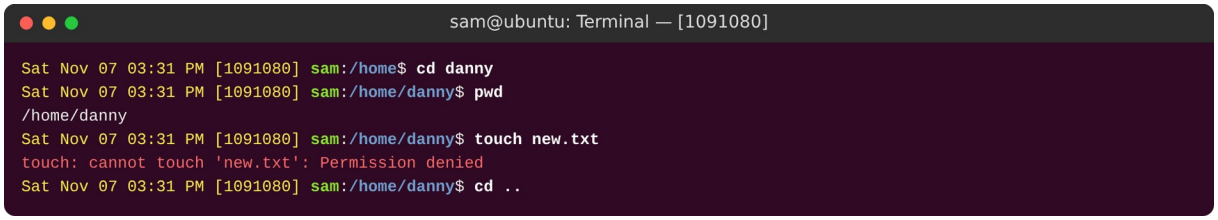


Figure 9: sam can enter danny's directory but cannot write in it

Others have `r-x` on danny's home folder, so sam can **enter** it (x) and **list** it (r). `touch new.txt` fails with "Permission denied" because creating a file changes the folder, and that needs **write (w)** on the folder. Others do not have it.

Step 3.3: As root, remove execute for others on danny's directory (chmod o-x)



Figure 10: As root, remove execute for others on danny's directory (chmod o-x)

Symbolic mode: `chmod o-x danny` removes (-) the execute right (x) from others (o). The string changes from `drwxr-xr-x` to `drwxr-xr--`. Other examples: `u+rwx`, `g-w`, `o=r`, `a+x`.

Step 3.4: Verify sam can no longer access danny's directory



Figure 11: Verify sam can no longer access danny's directory

Now sam gets "Permission denied" on `cd danny`. For a folder, **x means the right to enter it**. Without x you cannot go into the folder or reach anything inside it, even if r is still set.

2.4 Part 4 – Absolute (Octal) Permissions

Octal	Binary	Permission	Meaning
7	111	rwx	Read, write, execute
6	110	rw-	Read and write
5	101	r-x	Read and execute
4	100	r--	Read only
3	011	-wx	Write and execute
2	010	-w-	Write only
1	001	--x	Execute only
0	000	---	No permission

Table 1: Octal permission values

Step 4.1: Switch to danny and view current permissions

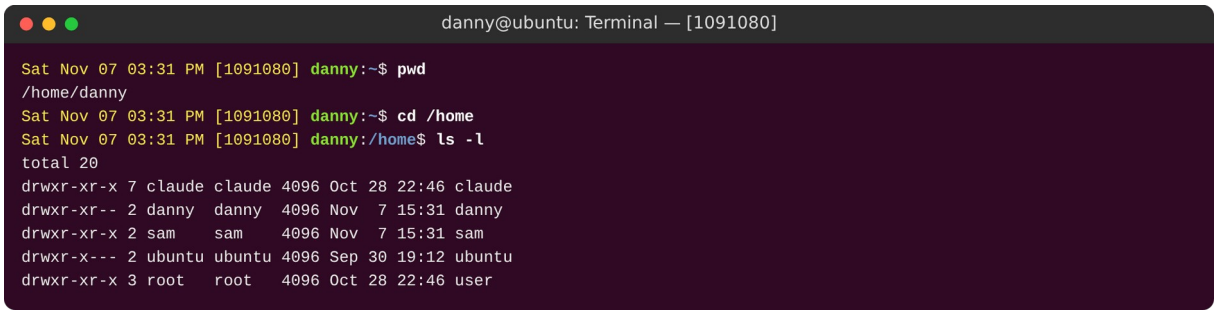


Figure 12: Switch to danny and view current permissions

Logged in as danny, the listing shows that others have `r--` on `/home/danny` (from the last step).

Step 4.2: Absolute mode: chmod 705 danny (owner rwx, group ---, others r-x)

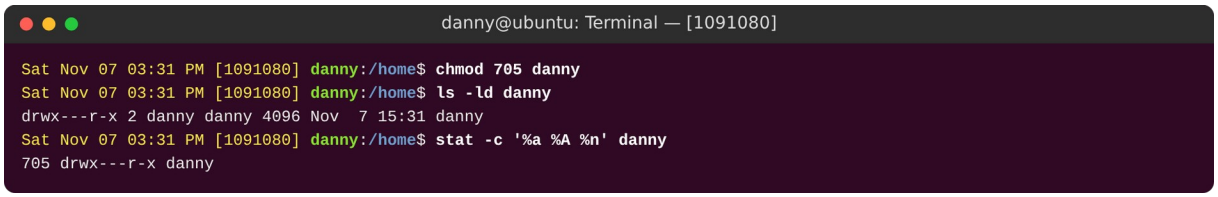


Figure 13: Absolute mode: chmod 705 danny (owner rwx, group ---, others r-x)

Absolute (octal) mode: each digit is the sum of r=4, w=2 and x=1. `705` means owner 7 (rwx), group 0 (---) and others 5 (r-x), so the result is `drwx---r-x`. `stat -c %a` confirms 705. Danny owns the folder, so he can change it without root. This is the "discretionary" part of DAC (the owner decides). **Interesting point:** the group digit is 0, so group members are blocked while others are allowed. Linux checks the owner first, then the group, then others, and uses only the first one that matches.

Step 4.3: Create test.txt in danny's directory

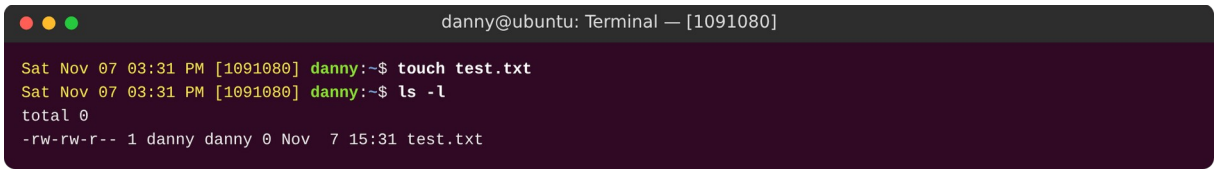


Figure 14: Create test.txt in danny's directory

`test.txt` is created as `-rw-rw-r--` (664). New files start at 666 and the umask (002) removes write for others. New files never get execute by default.

Step 4.4: As sam: read access to danny's directory and test.txt

```
sam@ubuntu: Terminal — [1091080]
Sat Nov 07 03:31 PM [1091080] sam:~$ cd /home
Sat Nov 07 03:31 PM [1091080] sam:/home$ ls -l
total 20
drwxr-xr-x 7 claude claude 4096 Oct 28 22:46 claude
drwx---r-x 2 danny  danny  4096 Nov  7 15:31 danny
drwxr-xr-x 2 sam    sam    4096 Nov  7 15:31 sam
drwxr-x--- 2 ubuntu ubuntu 4096 Sep 30 19:12 ubuntu
drwxr-xr-x 3 root   root   4096 Oct 28 22:46 user
Sat Nov 07 03:31 PM [1091080] sam:/home$ cd danny
Sat Nov 07 03:31 PM [1091080] sam:/home/danny$ ls -l
total 0
-rw-rw-r-- 1 danny danny 0 Nov  7 15:31 test.txt
Sat Nov 07 03:31 PM [1091080] sam:/home/danny$ cat test.txt && echo '(file readable - empty)'
(file readable - empty)
```

Figure 15: As sam: read access to danny's directory and test.txt

sam is one of the "others". He can enter (x) and list (r) danny's folder, and he can read `test.txt` because the file has `r--` for others.

Step 4.5: As sam: attempt to create a file in danny's directory

```
sam@ubuntu: Terminal — [1091080]
Sat Nov 07 03:31 PM [1091080] sam:/home/danny$ touch Sam.txt
touch: cannot touch 'Sam.txt': Permission denied
```

Figure 16: As sam: attempt to create a file in danny's directory

sam still cannot create `Sam.txt` because others have no `w` on the folder. Access is read-only, which is what the 705 setting is meant to do.

2.5 Part 5 – Exploring SUID, SGID and Sticky Bit (Self-Learning)

Step 5.1: SUID: create test directory, write and compile the C program (owner danny)

```
root@ubuntu: Terminal — [1091080]
Sat Nov 07 03:31 PM [1091080] root:~# cd /opt/suidlab
Sat Nov 07 03:31 PM [1091080] root:/opt/suidlab# echo 'TOP-SECRET: only danny can read this' > danny_secret.txt
Sat Nov 07 03:31 PM [1091080] root:/opt/suidlab# chown danny:danny danny_secret.txt && chmod 600 danny_secret.txt
Sat Nov 07 03:31 PM [1091080] root:/opt/suidlab# cat whoami_suid.c
#include <stdio.h>
#include <unistd.h>
#include <pwd.h>
static const char *nm(uid_t u){ struct passwd *p = getpwuid(u); return p ? p->pw_name : "?"; }
int main(void){
    uid_t r = getuid(), e = geteuid();
    printf("Real UID      : %d (%s)\n", r, nm(r));
    printf("Effective UID : %d (%s)\n", e, nm(e));
    FILE *f = fopen("/opt/suidlab/danny_secret.txt", "r");
    if (f) { char b[128]; if (fgets(b, sizeof b, f)) printf("Secret file   : %s", b); fclose(f); }
    else    printf("Secret file   : Permission denied\n");
    return 0;
}
Sat Nov 07 03:31 PM [1091080] root:/opt/suidlab# gcc whoami_suid.c -o whoami_suid
Sat Nov 07 03:31 PM [1091080] root:/opt/suidlab# chown danny:danny whoami_suid && chmod 755 whoami_suid
Sat Nov 07 03:31 PM [1091080] root:/opt/suidlab# ls -l
total 24
-rw------- 1 danny danny    37 Nov  7 15:31 danny_secret.txt
-rwxr-xr-x 1 danny danny 16344 Nov  7 15:31 whoami_suid
-rw-r--r-- 1 root  root    541 Nov  7 15:31 whoami_suid.c
```

Figure 17: SUID: create test directory, write and compile the C program (owner danny)

SUID test setup. A small C program prints the **real UID** (`getuid()`, the user who started it) and the **effective UID** (`geteuid()`, the identity Linux uses for permission checks). It also tries to read `danny_secret.txt`, which only danny can read (mode 600). The program file belongs to danny.

Step 5.2: SUID BEFORE: sam runs the program (runs as sam)

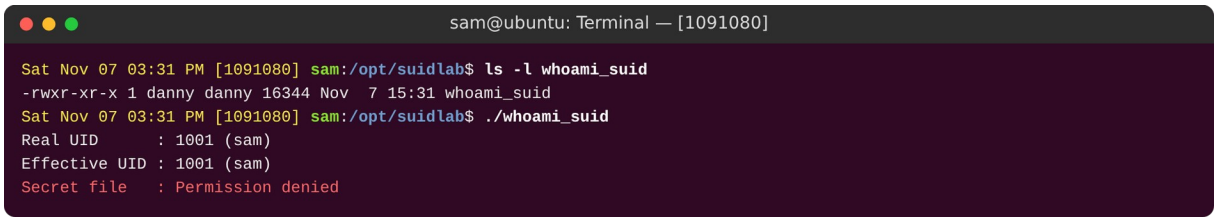


Figure 18: SUID BEFORE: sam runs the program (runs as sam)

Before SUID, the program runs as sam: the real UID and the effective UID are both 1001. It cannot read the secret file.

Step 5.3: Apply SUID with chmod u+s

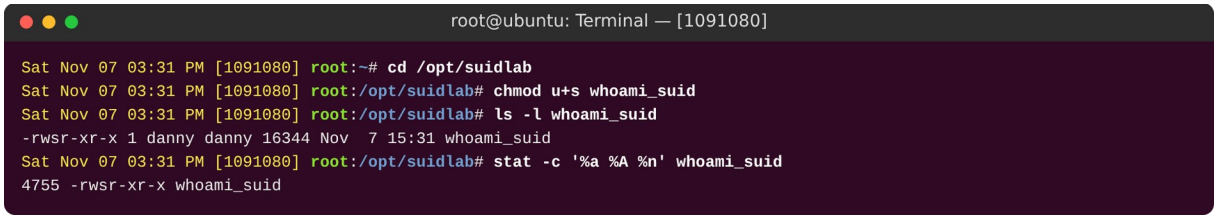


Figure 19: Apply SUID with chmod u+s

`chmod u+s` sets the SUID bit. The owner's x changes to `s` (`-rwsr-xr-x`) and the octal mode becomes **4755** (4 = SUID, 2 = SGID, 1 = sticky).

Step 5.4: SUID AFTER: sam runs the program (effective UID = danny)

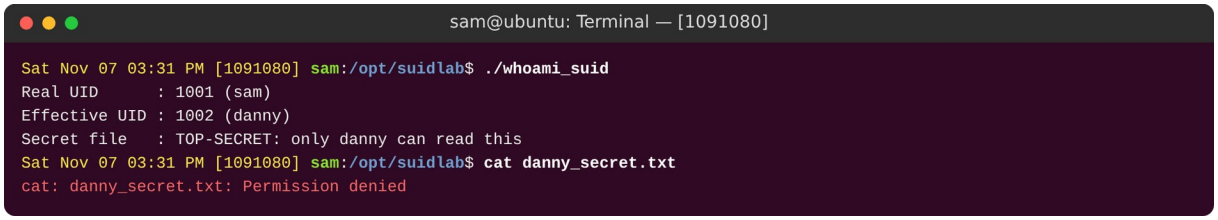


Figure 20: SUID AFTER: sam runs the program (effective UID = danny)

After SUID, the real UID is still sam, but the effective UID is **danny**, so the program can read danny's secret. If sam runs `cat` directly, he is still denied. So the extra power exists only **inside** the SUID program.

Step 5.5: Real-world SUID binaries and how auditors find them

```
root@ubuntu: Terminal — [1091080]

Sat Nov 07 03:31 PM [1091080] root:~# ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 64152 May 30  2024 /usr/bin/passwd
Sat Nov 07 03:31 PM [1091080] root:~# find /usr/bin -perm -4000 -type f 2>/dev/null | head
/usr/bin/sudo
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/su
/usr/bin/umount
```

Figure 21: Real-world SUID binaries and how auditors find them

Real systems use SUID for programs such as `passwd`, which must write to `/etc/shadow` for a normal user. SUID programs owned by root are a favourite target for attackers, so auditors list them with `find / -perm -4000` and compare the list with a known good one.

Step 5.6: SGID BEFORE: shared directory owned by DEVELOPERS, no SGID

```
root@ubuntu: Terminal — [1091080]

Sat Nov 07 03:31 PM [1091080] root:~# mkdir /srv/devshare
Sat Nov 07 03:31 PM [1091080] root:~# chgrp DEVELOPERS /srv/devshare
Sat Nov 07 03:31 PM [1091080] root:~# chmod 2770 /srv/devshare && chmod g-s /srv/devshare
Sat Nov 07 03:31 PM [1091080] root:~# ls -ld /srv/devshare
drwxrwx--- 2 root DEVELOPERS 4096 Nov  7 15:31 /srv/devshare
Sat Nov 07 03:31 PM [1091080] root:~# su sam -c 'touch /srv/devshare/before_sgid.txt'
Sat Nov 07 03:31 PM [1091080] root:~# ls -l /srv/devshare
total 0
-rw-rw-r-- 1 sam sam 0 Nov  7 15:31 before_sgid.txt
```

Figure 22: SGID BEFORE: shared directory owned by DEVELOPERS, no SGID

SGID before: `/srv/devshare` belongs to the group DEVELOPERS (mode 770). A file that sam creates gets sam's **own** group (`sam`), so danny cannot share it through the group.

Step 5.7: SGID AFTER: chmod g+s - new files inherit the DEVELOPERS group

```
root@ubuntu: Terminal — [1091080]

Sat Nov 07 03:31 PM [1091080] root:~# chmod g+s /srv/devshare
Sat Nov 07 03:31 PM [1091080] root:~# ls -ld /srv/devshare
drwxrws--- 2 root DEVELOPERS 4096 Nov  7 15:31 /srv/devshare
Sat Nov 07 03:31 PM [1091080] root:~# su sam -c 'touch /srv/devshare/after_sgid_sam.txt'
Sat Nov 07 03:31 PM [1091080] root:~# su danny -c 'touch /srv/devshare/after_sgid_danny.txt; mkdir /srv/devshare/subdir'
Sat Nov 07 03:31 PM [1091080] root:~# ls -l /srv/devshare
total 4
-rw-rw-r-- 1 danny DEVELOPERS    0 Nov  7 15:31 after_sgid_danny.txt
-rw-rw-r-- 1 sam   DEVELOPERS    0 Nov  7 15:31 after_sgid_sam.txt
-rw-rw-r-- 1 sam   sam           0 Nov  7 15:31 before_sgid.txt
drwxrwsr-x 2 danny DEVELOPERS 4096 Nov  7 15:31 subdir
```

Figure 23: SGID AFTER: chmod g+s - new files inherit the DEVELOPERS group

`chmod g+s` sets SGID on the folder (`drwxrws---`). New files from sam and danny now **get the DEVELOPERS group**, and new sub-folders also get SGID (`drwxrwsr-x`). The old file keeps the group `sam`, because SGID only affects new files. This is the usual way to make a shared team folder.

Step 5.8: Sticky BEFORE: world-writable /tmp/teststicky - danny deletes sam's file

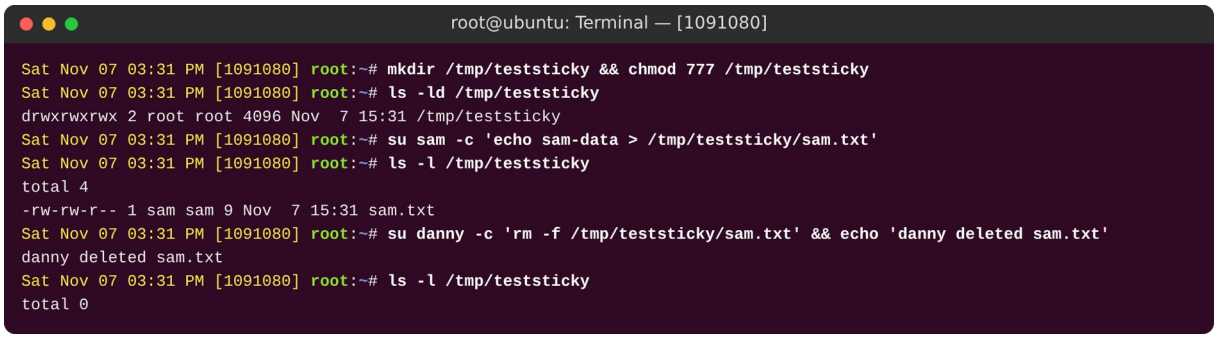


Figure 24: Sticky BEFORE: world-writable /tmp/teststicky - danny deletes sam's file

Sticky bit before: `/tmp/teststicky` is open to everyone (777). danny deletes sam's file without any problem, because deleting a file needs write permission on the **folder**, not on the file.

Step 5.9: Sticky AFTER: chmod +t - only owner or root can delete

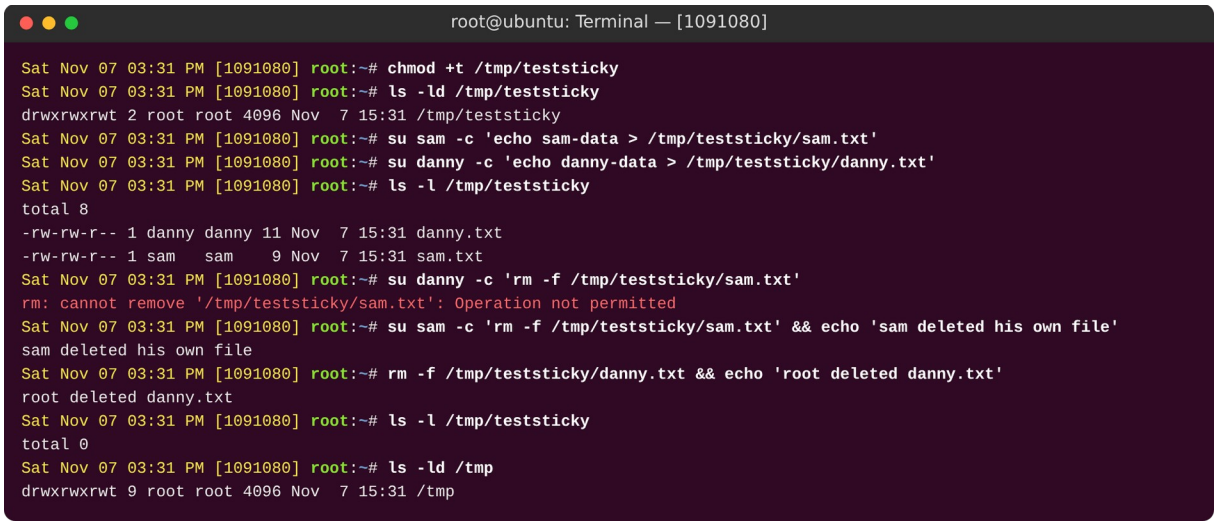


Figure 25: Sticky AFTER: chmod +t - only owner or root can delete

`chmod +t` sets the sticky bit (`drwxrwxrwt`, mode 1777). Now danny gets "Operation not permitted" when he deletes sam's file. Only the file owner (sam) or root can delete it. The system folder `/tmp` uses the same bit for the same reason.

2.6 Summary of Special Permission Bits

Bit	Octal	Set with	On files	On directories
SUID	4000	chmod u+s	Runs with the file owner's user ID	No effect on Linux
SGID	2000	chmod g+s	Runs with the file's group	New files get the folder's group
Sticky	1000	chmod +t	Ignored on Linux	Only the owner or root can delete or

				rename files

Table 2: Special permission bits compared

2.7 Critical Analysis – Part A

What worked well. DAC is simple and fast, and each owner manages their own data. Danny secured his folder with `chmod 705` without needing root. Groups together with SGID make it easy to share project files without opening them to everyone.

Weaknesses I saw.

- **The owner decides.** A careless owner can open files to everyone (`chmod 777`) and the system does not stop them. Mandatory Access Control (SELinux or AppArmor) adds system-wide rules that the owner cannot change.
- **Only three classes.** You cannot say "sam may read but Eve may not" when both are in "others". POSIX ACLs (`setfacl -m u:sam:rx`) solve this.
- **SUID is powerful and risky.** My test showed sam reading danny's secret through the program. If a SUID program owned by **root** has a bug (for example a buffer overflow), an attacker can get a full root shell. A real example is CVE-2021-4034 (PwnKit) in `pkexec`. Rules to follow: avoid your own SUID programs, mount user-writable disks with `nosuid`, and check regularly with `find / -perm -4000`.
- **Weak lab passwords.** `samPass` and `dannyPass` would fail a real password policy. In production I would use `pam_pwquality` (length and complexity), password ageing with `chage`, and account lockout with `pam_faillock`.

My own reflection. The most useful lesson was that permissions on a **folder** mean something different from permissions on a **file**. Whether you can delete someone else's file depends on the folder's w bit, and that is exactly why the sticky bit exists. I also saw that new Ubuntu versions are "secure by default" (home mode 0750 and yescrypt hashes), which is stronger than the older setup in the handout.

3. Part B – Getting Started with Keycloak Identity Provider

Keycloak is a free, open-source tool for Identity and Access Management. It gives Single Sign-On (one login for many apps) and can connect to other login systems. It uses the standards **OAuth 2.0**, **OpenID Connect (OIDC)** and **SAML 2.0**. Applications send users to Keycloak to log in and get back signed **JSON Web Tokens (JWT)**, so the applications never handle the passwords.

Key words: a **realm** is a separate space with its own users, roles and clients. A **user** is an identity that can log in. A **role** is a named permission given to users or service accounts. A **client** is an application that asks Keycloak to log in a user (or to log in itself).

Step 1: Running Keycloak using Docker

```
sudo su
docker run -d --name keycloak -p 8080:8080 \
  -e KC_BOOTSTRAP_ADMIN_USERNAME=Anas \
  -e KC_BOOTSTRAP_ADMIN_PASSWORD=1091080 \
  keycloak-local:26.8.2 start-dev
docker ps
```

What I ran: the lab uses the image `quay.io/keycloak/keycloak:latest`, but my machine could not reach quay.io. So I downloaded the official Keycloak 26.8.2 release from Maven Central (its checksum matched) and built a small local Docker image from it (`keycloak-local:26.8.2`, on top of Eclipse Temurin Java 21). I used the same command as the lab with `start-dev`, plus `-d` (run in the background) and `--name keycloak`. `-p 8080:8080` opens port 8080 on the computer. `KC_BOOTSTRAP_ADMIN_USERNAME` and `KC_BOOTSTRAP_ADMIN_PASSWORD` create the first temporary admin. `start-dev` is development mode (plain HTTP and a built-in test database) and must **never** be used in production. `docker ps` shows that the container is running.

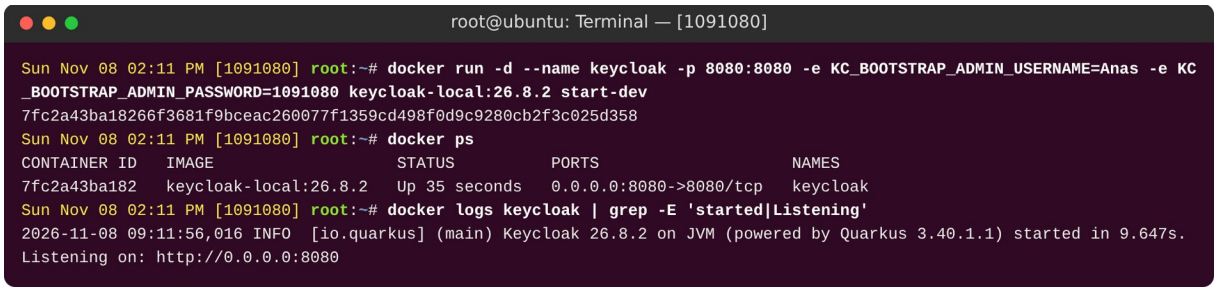


Figure 26: docker ps output showing the Keycloak container running

Step 2: Logging into the Keycloak Admin Console

```
Browser → http://localhost:8080/  →  username: Anas   password: 1091080
```

The admin console belongs to the **master** realm. Keycloak shows the user name in lower case (`anas`) because it always saves user names in lower case. The yellow banner says I am a temporary admin, which is normal for the first admin account.

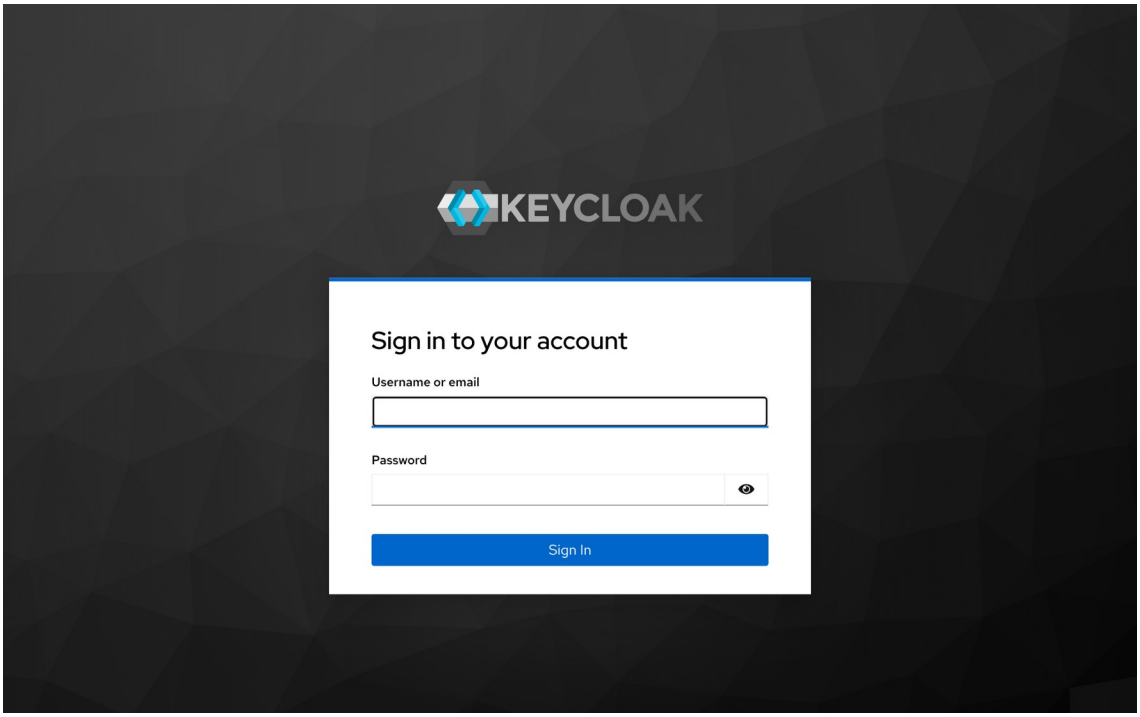


Figure 27: Keycloak admin login page

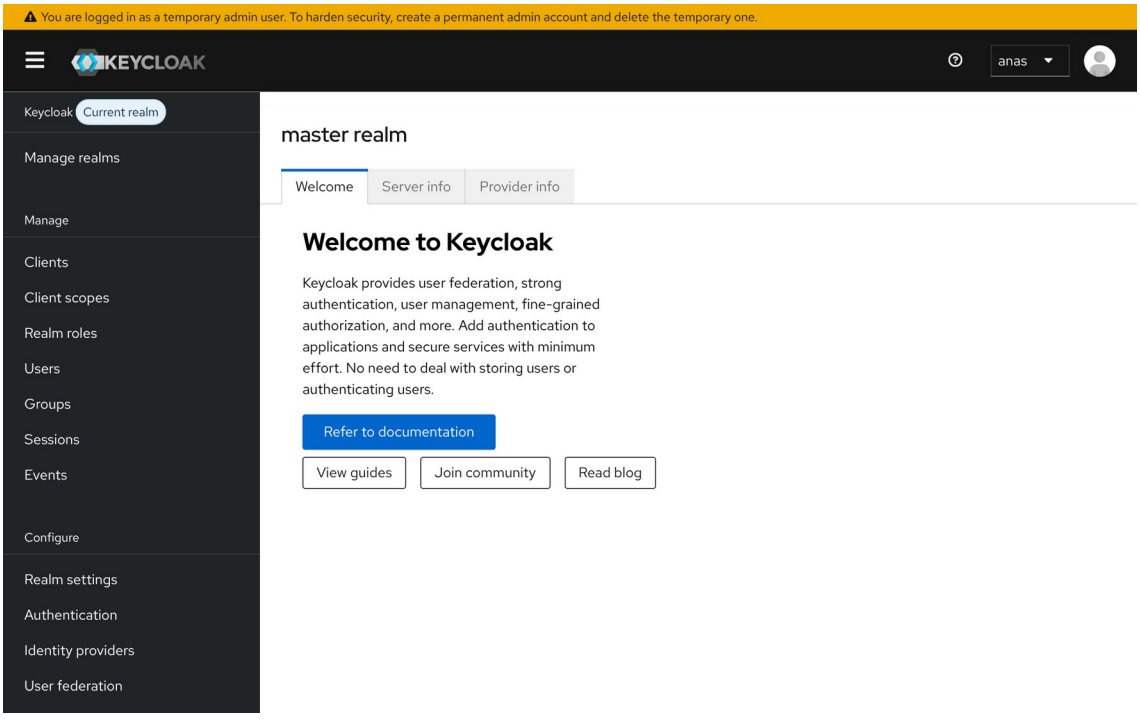


Figure 28: Admin console after successful login

Step 3: Configuring OTP (two-factor authentication) for the admin user

```
Users → Anas → Details → Required user actions: Configure OTP → Save
Sign out → sign in again → set up the authenticator (QR code, or the key under 'Unable
to scan?') → enter the 6-digit code
```

OTP adds a second step to the login: **something you have** (the phone) as well as **something you know** (the password). Keycloak uses **TOTP (RFC 6238)**: the QR code shares a secret key, and both sides use that key and the current 30-second time window to work out the same 6-digit code. If someone steals the admin password, they still cannot log in. On my lab machine I used the key shown under "Unable to scan?" and a small script to work out the 6-digit code (the same TOTP method as an authenticator app). Keycloak **refused** the same code when I used it a second time inside the same 30 seconds, which stops codes from being reused.

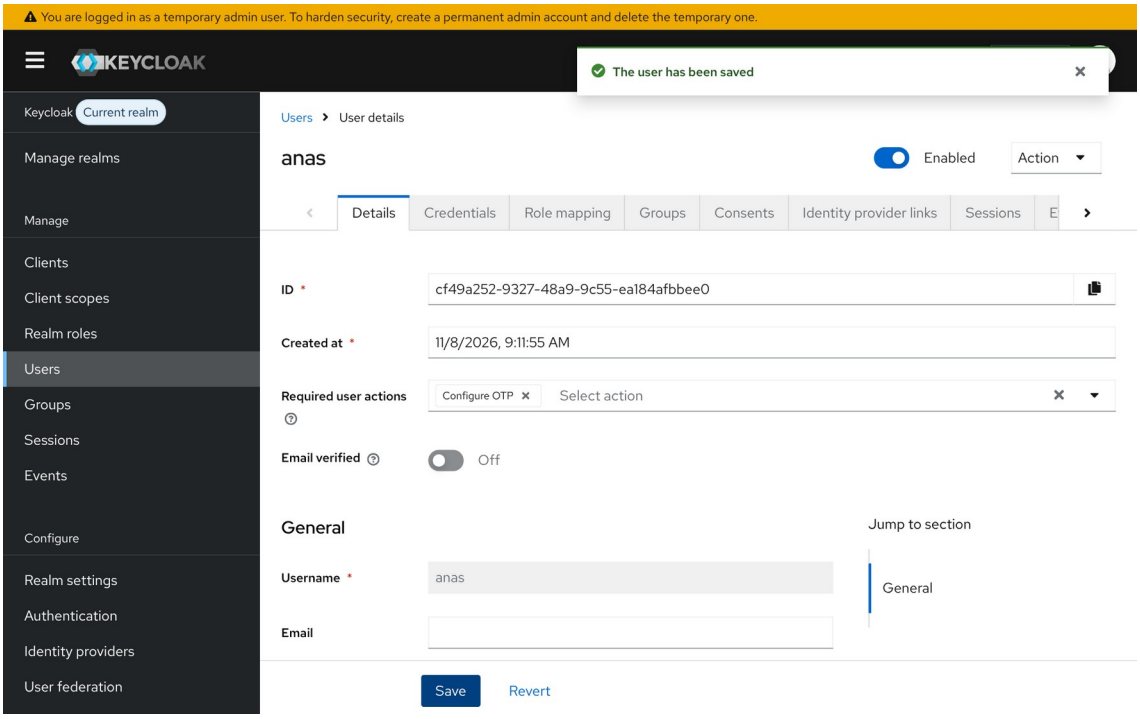


Figure 29: User details page with the 'Configure OTP' required action (saved)

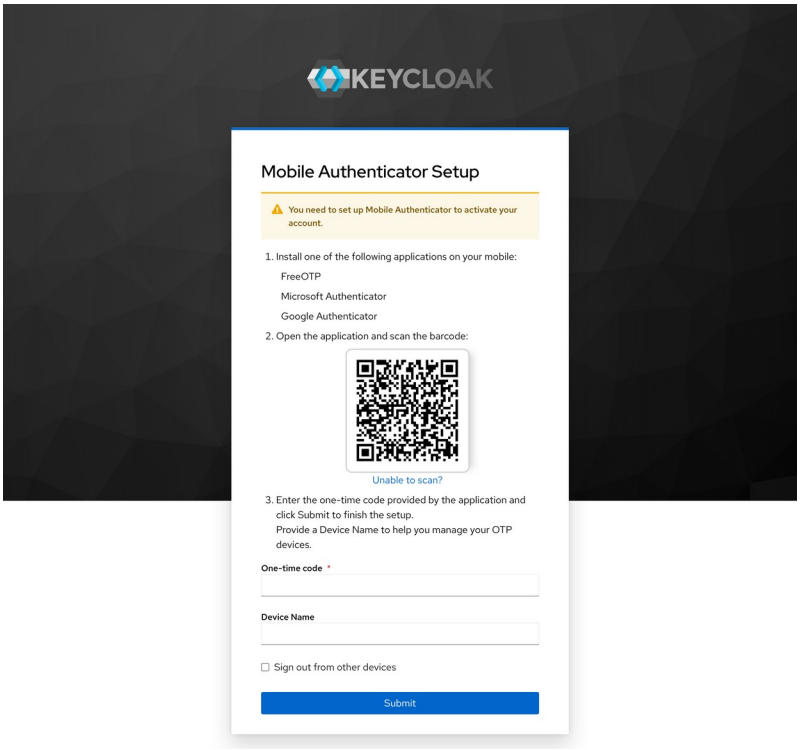


Figure 30: QR code / Mobile Authenticator Setup page

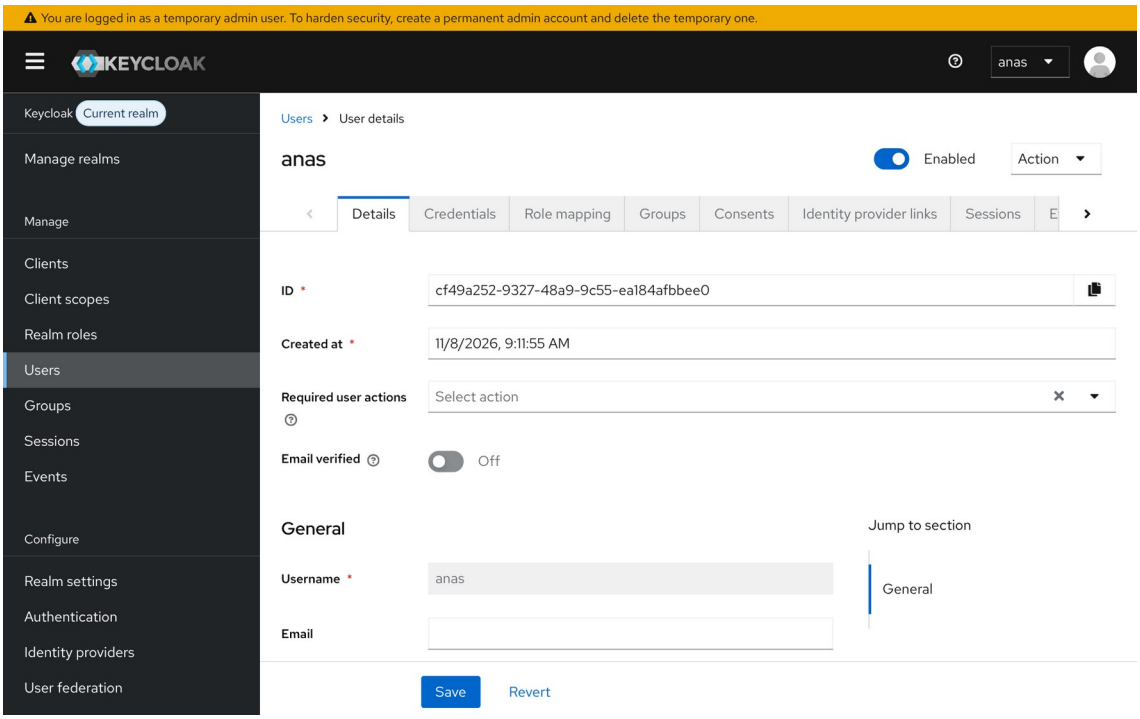


Figure 31: Successful login after entering the OTP (the Configure OTP action is now cleared)

Step 4: Creating a new realm MyApp080

```
Manage realms → Create realm → Realm name: MyApp080 → Create
```

Keeping applications out of the **master** realm follows the rule of least privilege. If an application user is hacked, they cannot reach the realm that controls Keycloak. Each realm has its own users, keys and settings. My realm is `MyApp080` (080 are the last three digits of my student ID).

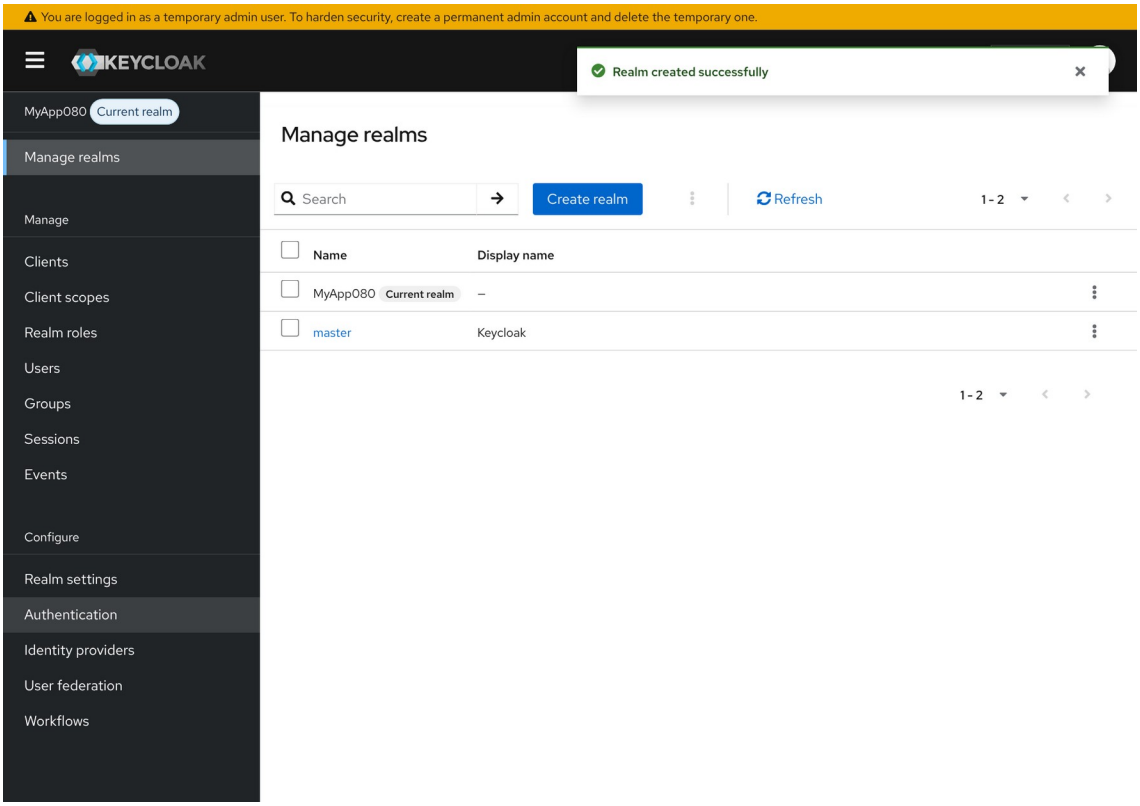


Figure 32: Realm MyApp080 created

Step 5: Creating a new user

```
MyApp080 → Users → Add user → Username: syed → Create
Credentials → Set password: 1091080, Temporary: OFF → Save
Test: http://localhost:8080/realms/MyApp080/account
```

The handout's screenshot note says "first name", but the instructions say "family name". I followed the instructions: my family name is Syed, so the user name is `syed`. I set the password to my student ID and turned **Temporary** off, so Keycloak does not force a password change. At the first login, Keycloak asked me to complete my profile (email, first name and last name) and then opened the account page. This shows that the user can log in to the new realm.

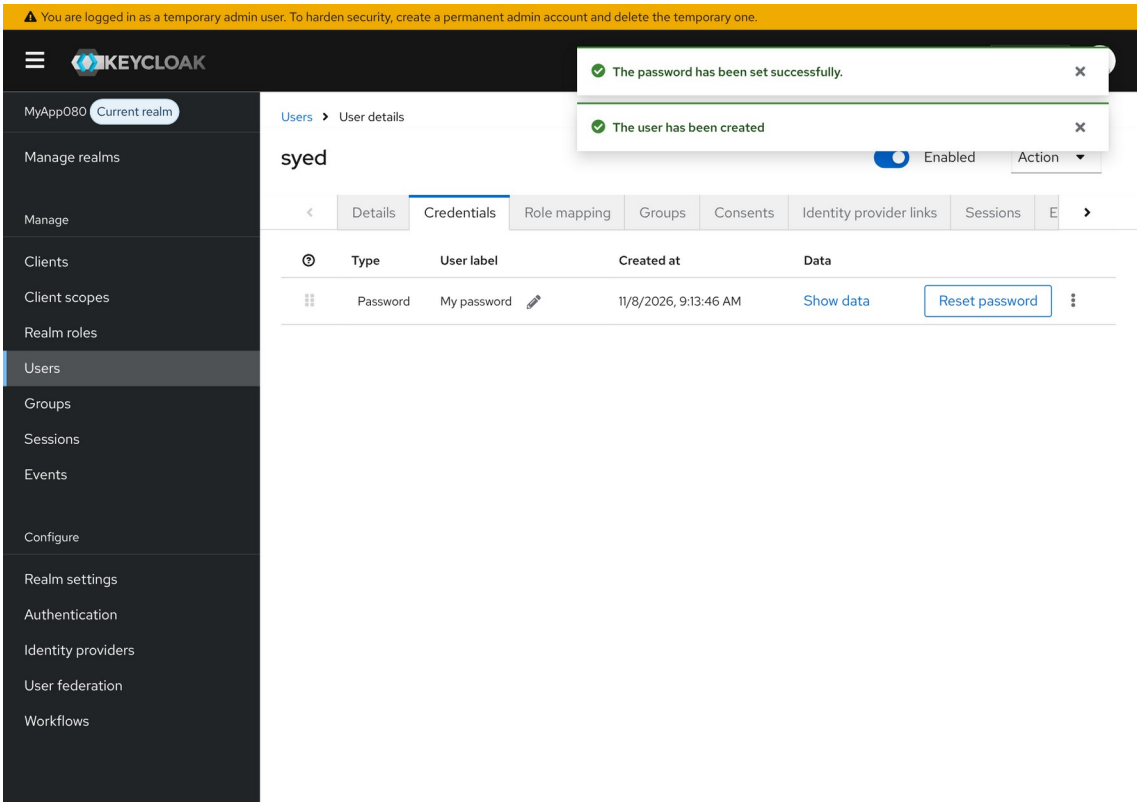


Figure 33: New user created in realm MyApp080

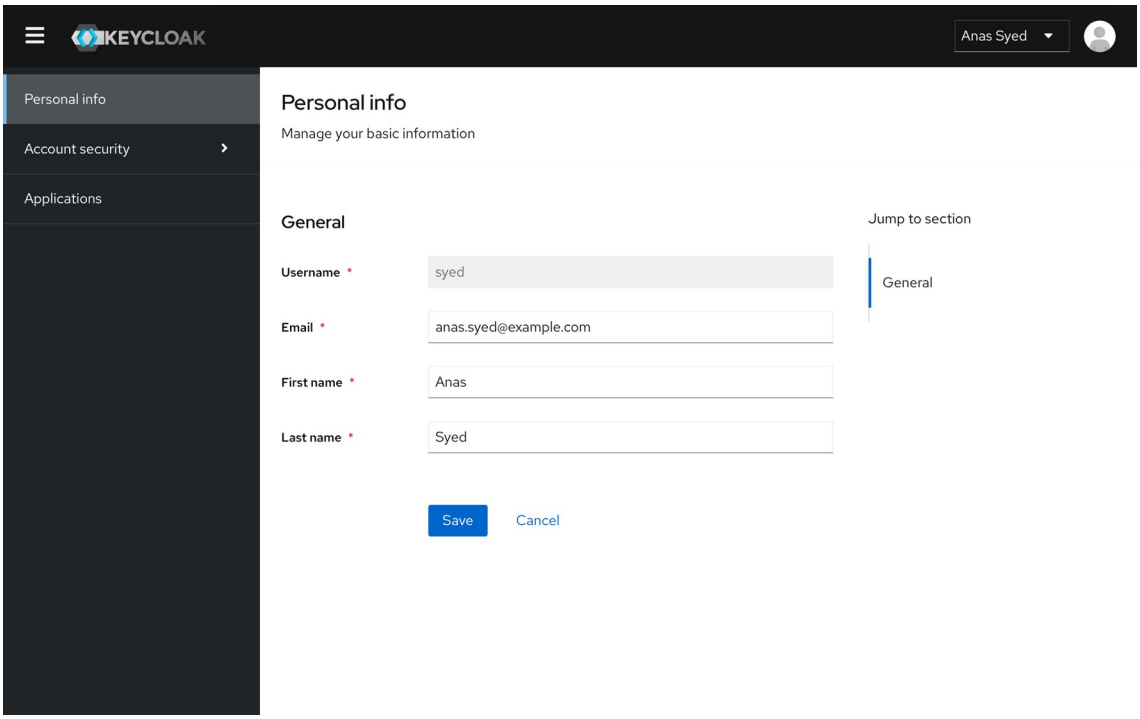


Figure 34: Successful login to the realm account page

Step 6: Creating a public client my-client080

```
Clients → Create client → Client type: OpenID Connect → Client ID: my-client080
Client authentication: OFF · Standard flow: ON · Direct access grants: ON
```

```
Valid redirect URIs: https://www.keycloak.org/app/*   Web origins:
https://www.keycloak.org → Save
```

A **public** client (a browser or phone app) cannot keep a secret safely, so it has no client secret. Its safety comes from checking the **redirect URI** exactly, and from PKCE. **Web origins** controls CORS, so only the page at keycloak.org can call the token address from a browser. Keycloak turns **Direct access grants** off by default, so I switched it on as the lab says.

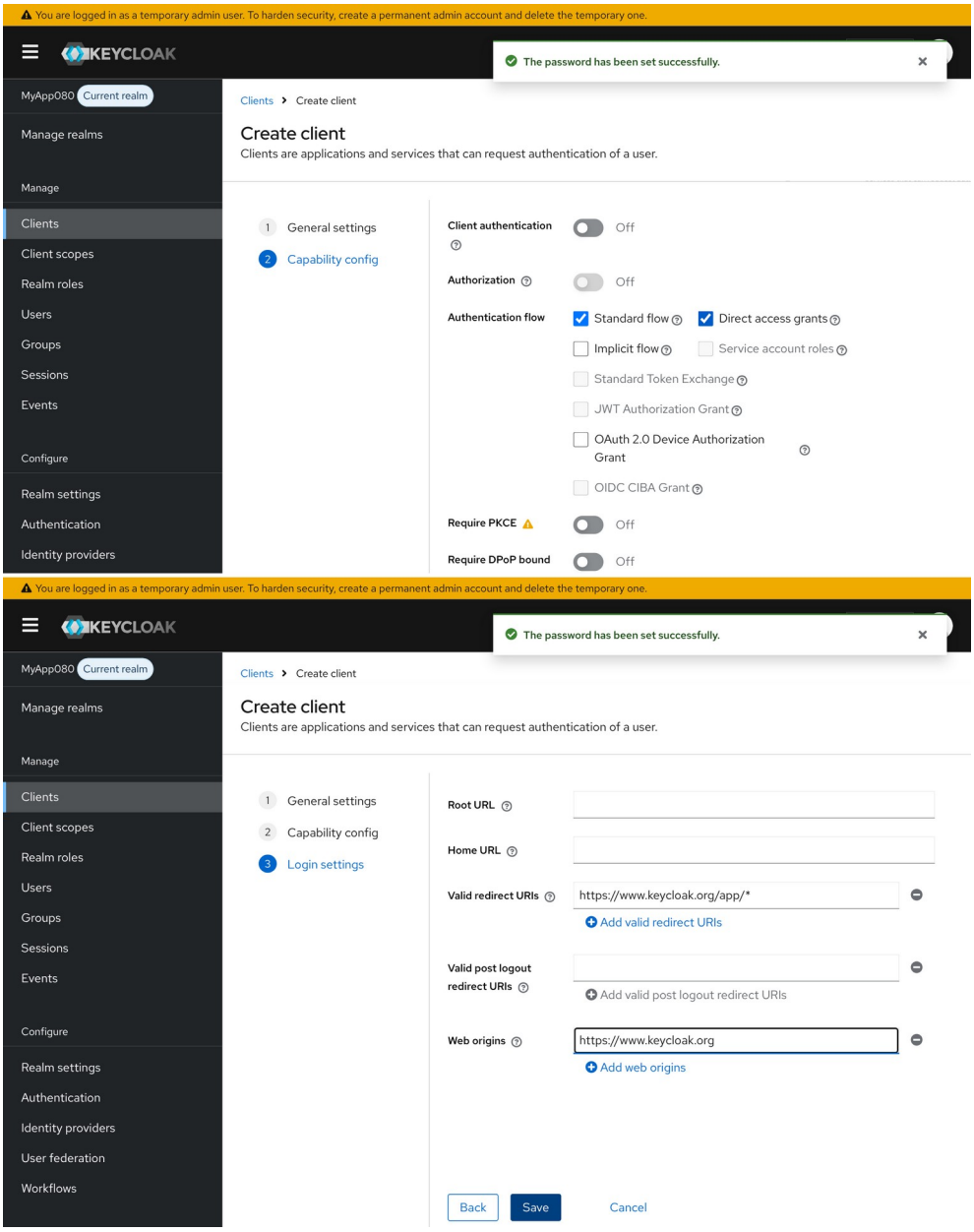


Figure 35: Public client my-client080: capability settings (top) and login settings (bottom)

Step 7: Testing the public client with the Keycloak demo application

```
https://www.keycloak.org/app/ → Realm: MyApp080, Client: my-client080 → Sign in
Log in as syed → read the /token request and response (Network tab, filter 'token')
```

This uses the **Authorization Code flow**: the browser goes to Keycloak, the user logs in there, and Keycloak sends back a short-lived code. The app swaps the code at `/token` for an `access_token`, an `id_token` and a `refresh_token`. **Note:** my lab machine could not reach https://www.keycloak.org/app/, so the browser opened a small local copy of the demo page at the same address. The Keycloak login page, the redirect and the token request and response are real. Figure 37 shows the captured token request.

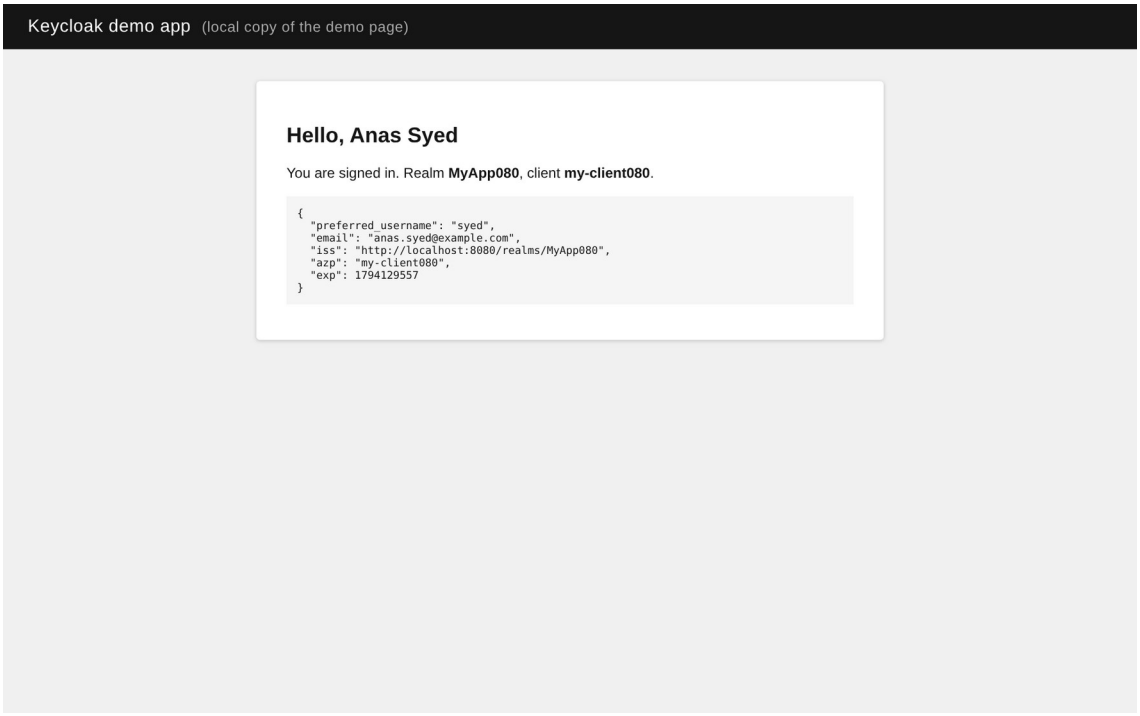


Figure 36: Test page signed in as the new user (local copy of the Keycloak demo page)

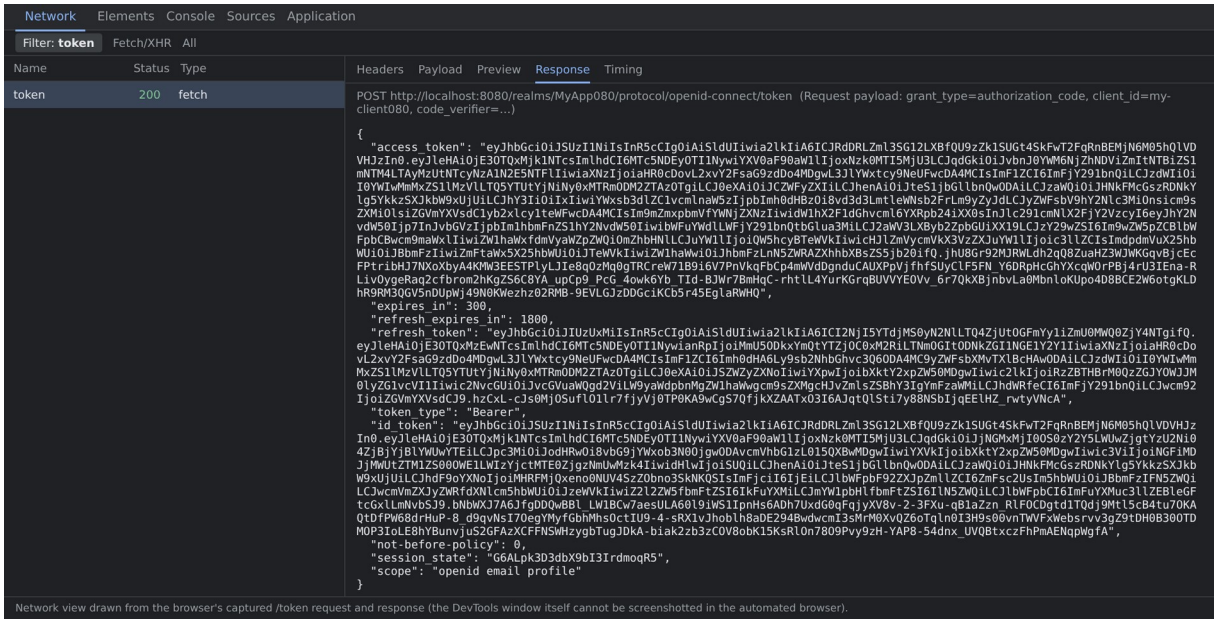


Figure 37: Network view of the /token request and response (drawn from the captured browser traffic)

Step 8: Creating a confidential client my-Confidential-Client080

```
Clients → Create client → Client ID: my-Confidential-Client080
```

```
Client authentication: ON · Direct access grants: ON · Service accounts roles: ON →
Save → Credentials tab → copy Client secret
```

A **confidential** client is a server-side application that can keep a secret. Turning on **Service accounts roles** lets the client get tokens for **itself** (machine to machine) with the Client Credentials grant. Keycloak makes the client secret automatically and shows it on the Credentials tab.

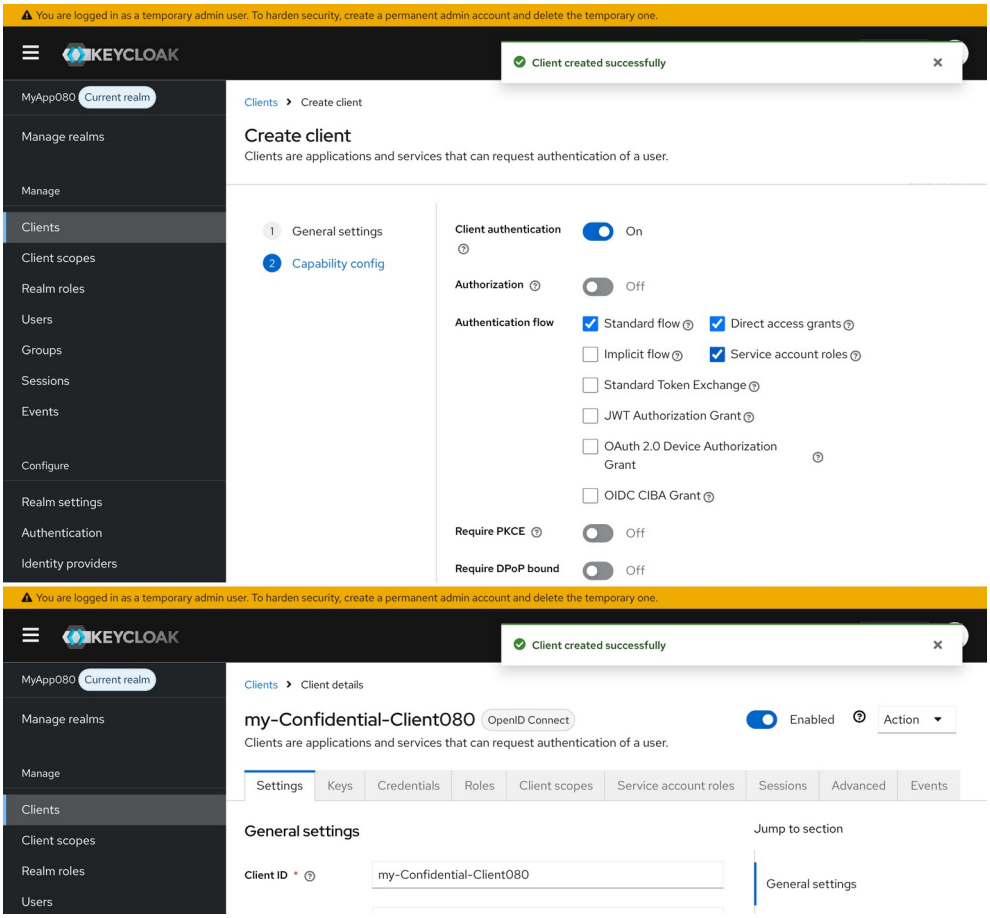


Figure 38: Confidential client my-Confidential-Client080: capability settings (top) and the created client (bottom)

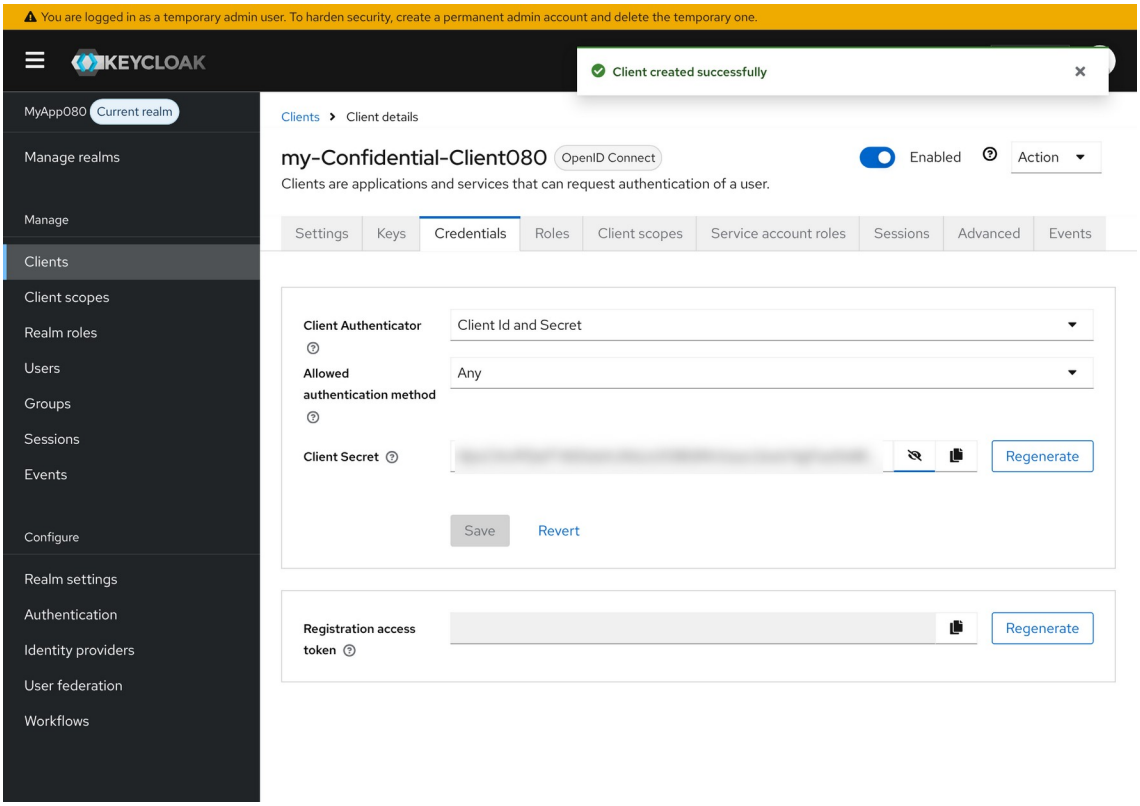


Figure 39: Credentials tab with client secret

Step 9: Requesting a token with curl and inspecting the JWT

```
curl -L -X POST
'http://localhost:8080/realms/MyApp080/protocol/openid-connect/token' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data-urlencode 'client_id=my-Confidential-Client080' \
  --data-urlencode 'client_secret=******** (hidden)' \
  --data-urlencode 'grant_type=client_credentials'
```

Correction to the handout: the given command has a space after `client_secret=`, which makes the login fail with `invalid_client`, so I left the space out. Also, the Client Credentials grant gives a token for the client's **service account** (`preferred_username: service-account-my-confidential-client080`), not for the user from Step 5.

Reading the JWT: a JWT has three parts joined by dots: `header.payload.signature` (each part is Base64URL text). The **header** shows `alg: RS256` and `kid` (which signing key was used). The **payload** holds the claims: `iss` (the realm address), `exp` and `iat` (expiry time and issue time, 5 minutes apart by default), `azp` and `client_id` (which client asked), `realm_access.roles` and `scope`. The **signature** is checked with the realm's public key from `/realms/MyApp080/protocol/openid-connect/certs`. Anyone can **read** a JWT, so secrets must never be put inside it; the signature protects it from changes. The token in Figure 41 was decoded locally because jwt.io could not be reached.

root@ubuntu: Terminal — [1091080]

```
Sun Nov 08 02:14 PM [1091080] root:~# curl -L -X POST 'http://localhost:8080/realms/MyApp080/protocol/openid-connect/token' \\
  -H 'Content-Type: application/x-www-form-urlencoded' \\
  --data-urlencode 'client_id=my-Confidential-Client080' \\
  --data-urlencode 'client_secret=[redacted]' \\
  --data-urlencode 'grant_type=client_credentials'
{"access_token":"eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJRdDRLZml3SG12LXBfQU9zZk1SUGt4SkFwT2FqRnBEMjN6M05hQlVDVHJzIn0.eyJleHAiOjE3OTQxMjk1NTgsImlhdCI6MTc5NDEyOTI1OCwianRpIjoidHJydGNjOmFiZTRlMzdkLTcwODMtMjlkYy0yMDFmLTQ1NDRjODFlNWNjZSIsImlzcyI6Imh0dHA6Ly9sb2NhbGhvc3Q6ODA4MC9yZWFsbXMvTXlBcHAwODAiLCJhdWQiOiJhY2NvdW50Iiwic3ViIjoiYWRiMTg4OTMtNjgwZC00YmY4LTg2M2QtZjNlMmVhMmRhZDg4IiwidHlwIjoiQmVhcmVyIiwiYXpwIjoibXktQ29uZmlkZW50aWFsLUNsaWVudDA4MCIsImFjciI6IjEiLCJyZWFsbV9hY2Nlc3MiOnsicm9sZXMiOlsiZGVmYXVsdC1yb2xlcy1teWFwcDA4MCIsIm9mZmxpbmVfYWNjZXNzIiwidW1hX2F1dGhvcml6YXRpb24iXX0sInJlc291cmNlX2FjY2Vzcyi6eyJhY2NvdW50Ijp7InJvbGVzIjpbIm1hbmFnZS1hY2NvdW50IiwibWFuYWdlLWFjY291bnQtbGlua3MiLCJ2aWV3LXByb2ZpbGUiXX19LCJzY29wZSI6ImVtYWlsIHByb2ZpbGUiLCJlbWFpbF92ZXJpZmllZCI6ZmFsc2UsImNsaWVudEhvc3QiOiIxNzIuMTcuMC4xIiwicHJlZmVycmVkX3VzZXJuYW1lIjoic2VydmljZS1hY2NvdW50LW15LWNvbmZpZGVudGlhbC1jbGllbnQwODAiLCJjbGllbnRBZGRyZXNzIjoiMTcyLjE3LjAuMSIsImNsaWVudF9pZCI6Im15LUNvbmZpZGVudGlhbC1DbGllbnQwODAifQ.Zqvtmnr1w3ZIIOwz9oPdAcjA4hq7POq7irnkRgNyuoZBtpQwtIJ2MAx-wT6R9GMOAQf2hqXwJlA7p884YbDjXgSUxx5jL6G2E9l3T3sAiFBaa48_8KNXZ9VRL1a7cxZxlPc8UU623uPRUEWNeGFw5g9pnp5r9rgcWbVurmRzQtb0WDFabg9pPV3qr8TY6P4WABiK1h5mONTX3kPAnnAlcWXzcBkmqWA7uJnoikthHwfYajJrPVBLQAMGmkOxLnoUzBcwwe23DhtqMbkj32Qwwm9rWMm_ieRXs4dquOAHDACklIKujxSW2B-QY2ADmaBwlpPXTBqrplJi4mA412pGiA","expires_in":300,"refresh_expires_in":0,"token_type":"Bearer","not-before-policy":0,"scope":"email profile"}
```

Figure 40: curl request and successful token response

Decoded JWT – access token for **my-Confidential-Client080** (MyApp080)

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "Qt4KfiwHmv-p_AOsfMRPkxJApOajFpD23z3NaBUCTrs"
}
```

SIGNATURE (Base64URL, first 60 characters)

```
Zqvtmnr1w3ZIIOwz9oPdAcjA4hq7POq7irnkRgNyuoZBtpQwtIJ2MA
x-wT6R…
```

exp = 2026-11-08 09:19:18 UTC
iat = 2026-11-08 09:14:18 UTC
Lifetime = 300 seconds (5 minutes)

PAYLOAD: DATA (claims)


```
{
  "exp": 1794129558,
  "iat": 1794129258,
  "jti": "trrtcc:abe4e37d-7083-29dc-201f-4544c81e5cce",
  "iss": "http://localhost:8080/realms/MyApp080",
  "aud": "account",
  "sub": "adb18893-680d-4bf8-863d-f3e2ea2dad88",
  "typ": "Bearer",
  "azp": "my-Confidential-Client080",
  "acr": "1",
  "realm_access": {
    "roles": [
      "default-roles-myapp080",
      "offline_access",
      "uma_authorization"
    ]
  },
  "resource_access": {
    "account": {
      "roles": [
        "manage-account",
        "manage-account-links",
        "view-profile"
      ]
    }
  },
  "scope": "email profile",
  "email_verified": false,
  "clientHost": "172.17.0.1",
  "preferred_username": "service-account-my-confidential-client080",
  "clientAddress": "172.17.0.1",
  "client_id": "my-Confidential-Client080"
}
```


Decoded locally from the real token returned by Keycloak (jwt.io could not be reached from this machine).

Figure 41: Decoded JWT (header and payload), decoded locally

Step 10: Gmail login via Google identity provider (Self-Learning)

```
Google Cloud Console → APIs & Services → OAuth consent screen (External, add test user)
Credentials → Create OAuth client ID → Web application
  Authorized redirect URI:
http://localhost:8080/realms/MyApp080/broker/google/endpoint
Keycloak → MyApp080 → Identity providers → Google → paste Client ID + Client secret → Add
Test: http://localhost:8080/realms/MyApp080/account → 'Sign in with Google'
```

This is **identity brokering**. Keycloak hands the login over to Google using OIDC. On the first login, Keycloak creates a user that is linked to the Google account. Users do not need another password, and the

application still gets Keycloak tokens. Google allows `http://localhost` redirect addresses for testing, but a real website must use HTTPS.

3.1 Critical Analysis – Part B

- **One central login** means applications do not store passwords, and it gives SSO, multi-factor login and one place to cut off a user. The downside is that Keycloak becomes a valuable target and a single point of failure, so it needs backup servers, updates and strong admin MFA.
- **Dev mode is not safe by design.** It uses plain HTTP and a test database, and the admin password was typed in the command line, where `docker inspect` and the shell history can show it. Production needs HTTPS, a real database, secret storage and the `start` command.
- **Bearer tokens** work for anyone who holds them. Short lifetimes (`exp`), HTTPS only, and checking the audience (`aud`) lower the risk if a token is stolen. In my test the token was valid for only 300 seconds.
- **My own reflection:** the clearest lesson was the difference between public and confidential clients: who can keep a secret decides which login flow is safe. I also learned that small details matter. Keycloak switches off Direct access grants by default, and it refused a one-time code that was used twice.

4. Conclusion

Part A showed access control at the operating-system level: users, groups, hashed passwords, symbolic and octal permissions, and the special bits SUID, SGID and sticky. Part B moved access control to the application level with Keycloak: realms, users, an admin protected by OTP, public and confidential clients, and signed JWT tokens. Together they show "defence in depth": least privilege in the operating system and strong central login in the application.

References

1. Kerrisk, M. (2010). The Linux Programming Interface. No Starch Press. Ch. 9 (Process Credentials) & Ch. 15 (File Attributes).
2. Linux man-pages project. chmod(1), adduser(8), usermod(8), shadow(5), credentials(7). https://man7.org/linux/man-pages/
3. Canonical. (2021). Ubuntu 21.04 release notes – private home directories (HOME_MODE 0750). https://discourse.ubuntu.com/
4. Qualys Research Team. (2022). PwnKit: Local Privilege Escalation Vulnerability in polkit's pkexec (CVE-2021-4034).
5. Keycloak. (2026). Server Administration Guide & Getting Started with Docker. https://www.keycloak.org/documentation
6. Hardt, D. (2012). RFC 6749 – The OAuth 2.0 Authorization Framework. IETF.
7. Jones, M., Bradley, J., Sakimura, N. (2015). RFC 7519 – JSON Web Token (JWT). IETF.
8. M'Raihi, D. et al. (2011). RFC 6238 – TOTP: Time-Based One-Time Password Algorithm. IETF.